t: 문장 → length = n

p: 검색하려는 단어 → length = m

t: T T T A A C C A . . index: i

p: T T A T T T C T . . index: j

↑

idx=0

while 반복 → 언제까지? t의 문장을 벗어나면 안됨 ⇒ $i < n$ and $j < m$

if t[i] == p[i] : → t의 첫번째와 p의 첫번째가 같으면

i+=1
j+=1 → i와 j를 증가시켜서 다시 비교 → 같으면 계속증가.

else : t[i] 와 p[j]가 다르면.

i = i - j + 1 : 현재 문장의 위치에서 불일치가 발생할때까지 이동한 j 만큼 돌아가고 2 다음 문장부터 탐색시작

j=0 → j는 0으로 돌아와서 검색하려는 단어의 처음부터 다시 비교

문장 속에서 패턴을 찾았을때 고려

count=0 → count는 while문 밖에서 선언해줘야 함

if j==M:

count +=1

j=

```
p = "is" # 찾을 패턴
t = "This is a book~!" # 전체 텍스트
M = len(p) # 찾을 패턴의 길이
N = len(t) # 전체 텍스트의 길이

def BruteForce(p, t) :
    i = 0 # t의 인덱스
    j = 0 # p의 인덱스
    while j < M and i < N :
        if t[i] != p[j] :
            i = i - j
            j = -1
        i = i + 1
        j = j + 1
    if j == M : return i - M # 검색 성공
    else : return -1 # 검색 실패
```

→ pattern과 text가 다를때 위에서는 i = i - j + 1, j=0 을 했지만

→ 기본적으로 i와 j가 계속 증가하여 탐색

이 코드에서는 i와 j가 계속 1씩 증가하는 코드이기 때문에 1씩 더 빼줘서 i = i - j, j=-1 을 하고

→ if 문 수행후 아래에서 다시 1씩 더해줄거라 1씩 더 빼줌

→ j== M이 되어버리면 t속에서 p를 찾았때 성공.

→ i - M하면 pattern 비교를 시작한 위치의 index가 나옴

```python
p='is'
t='This is book~! is'

def bf(p,t):
    M=len(p)
    N=len(t)
    i,j=0,0 #i는 t의 인덱스, j는 p의 인덱스
    count = 0
    while i<N and j<M:
        if t[i]!=p[j]: #텍스트와 패턴이 다를 때
            i=i-j #패턴과 텍스트가 달랐던 위치로 돌아가서 다시 탐색
            j=-1
        i+=1 #계속 해서 i와 j가 증가하며 텍스트와 패턴을 비교
        j+=1

        if j==M: #문장속에서 패턴을 찾은 경우
            count+=1 #count를 올려주고
            j=0  #j는 0으로 돌아가서 텍스트에서 패턴을 찾아낸 다음 인덱스에서부터 다시 탐색 시작
```

j=0 으로 돌아가서 t[0]='i' 부터 비교하고

i는 그대로 두어 해서 패턴을 찾아낸 그 다음 index부터 계속해서 탐색시작

```python
    return count

print(bf(p,t)) #3
```